UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 05

Bit Manipulation

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 ¿Por qué operar directamente sobre bits? 2

2 Visualizar bits: `bitset<N>` 2
2.1 Literales en distintas bases 2

3 Patrones especiales 3
3.1 Todos los bits apagados: `x = 0` 3
3.2 Todos los bits encendidos: `x = -1` 3
3.3 Un solo bit encendido: `1 « k` 3

4 Desplazamientos 4
4.1 Desplazamiento izquierdo « 4
4.2 Desplazamiento derecho » 4

5 Operadores lógicos bit a bit 4
5.1 NOT bit a bit (~) 4
5.2 AND bit a bit (&) 5
5.3 OR bit a bit (|) 5
5.4 XOR bit a bit (^) 6

6 Combinando operadores 6

7 Ejercicios 7
7.1 Desplazamientos 7
7.2 NOT, AND, OR y XOR 7
7.3 Combinando operadores 8

8 Resumen 8

¿Por qué operar directamente sobre bits?

Ya sabemos que toda la información en una computadora son bits. Hasta ahora los hemos leído y convertido entre bases, pero sin tocarlos. Esta sesión vamos a **manipularlos directamente** desde C++.

¿Para qué sirve eso?

- **Velocidad**: las operaciones de bits son las más baratas que existen en un procesador — cuestan un solo ciclo de reloj, sin importar si el operando tiene 8 o 64 bits.
- **Compacidad**: puedes empacar 32 valores booleanos en un solo `int`, en lugar de usar 32 variables separadas.
- **Programación de sistemas**: drivers, protocolos de red, compresión, cifrado y programación competitiva hacen uso intensivo de estas operaciones.

Los operadores que vamos a estudiar son:

Operador	Símbolo	Qué hace
Desplazamiento izquierdo	`«`	Mueve todos los bits a la izquierda
Desplazamiento derecho	`»`	Mueve todos los bits a la derecha
NOT bit a bit	`~`	Invierte cada bit
AND bit a bit	`&`	1 solo si ambos bits son 1
OR bit a bit	`|`	1 si al menos un bit es 1
XOR bit a bit	`^`	1 si los bits son distintos

Visualizar bits: `bitset<N>`

Antes de operar, necesitamos **ver** los bits. C++ incluye `bitset<N>` en la biblioteca `<bitset>`: convierte cualquier entero a su representación en exactamente N bits.

```
#include <bitset>
using bin8 = bitset<8>;

int x = 30;
cout << bin8(x);   // imprime: 00011110
```

$30 = 16 + 8 + 4 + 2$, y los cuatro bits bajos encendidos confirman eso.

`bitset` es solo para **visualización**: no cambia el valor almacenado ni el tipo de la variable.

• Literales en distintas bases

C++14 permite escribir el mismo valor en decimal, binario (`0b`) o hexadecimal (`0x`). Para el compilador son idénticos.

Forma	Ejemplo	Valor
Decimal	`30`	30
Binario	`0b00011110`	30
Hexadecimal	`0x1E`	30

Usar `0b` o `0x` no cambia nada en tiempo de ejecución — es solo una comodidad para que el código sea más legible cuando trabajamos con máscaras o valores relacionados con bits.

Un `char` también es un entero de 8 bits. Puedes inicializarlo con `char c = 0b01001101;` (que resulta ser la `'M'`) y usar `bitset` para ver sus bits igual que con un `int`.

Patrones especiales

Tres patrones aparecen constantemente en cualquier código que manipule bits. Vale la pena memorizarlos.

- **Todos los bits apagados: `x = 0`**

  Obvio pero importante: el valor 0 tiene todos sus bits en 0.

- **Todos los bits encendidos: `x = -1`**

  En complemento a 2, $-1 = 2^n - 1$, que en binario es todo 1s sin importar cuántos bits tenga el tipo. Es la forma idiomática en C++ de poner a 1 todos los bits de una variable.

```
int x = -1;   // 11111111 11111111 11111111 11111111  (32 bits)
```

- **Un solo bit encendido: `1 « k`**

  `1 « k` tiene exactamente el bit k encendido y todos los demás apagados. Vale 2^k.

Expresión	Valor	Binario (8 bits)
`1 « 0`	1	00000001
`1 « 1`	2	00000010
`1 « 2`	4	00000100
`1 « 3`	8	00001000
`1 « 4`	16	00010000
`1 « 5`	32	00100000
`1 « 6`	64	01000000
`1 « 7`	128	10000000

Estas expresiones se llaman **máscaras de un bit** y son el ingrediente básico de casi toda técnica de bit manipulation.

Desplazamientos

• Desplazamiento izquierdo «

`x « k` mueve todos los bits de `x` hacia la izquierda k posiciones. Los bits que salen por la izquierda se **pierden**; los huecos de la derecha se llenan con **0**.

Efecto aritmético: `x « k` $= x \times 2^k$ (siempre que no haya desbordamiento).

6 (antes):	0 0 0 0 0 1 1 0
Desplazar 2:	←←
24 (después):	0 0 0 1 1 0 0 0

$6 \times 4 = 24$. ✓

• Desplazamiento derecho »

`x » k` mueve todos los bits hacia la derecha k posiciones. Los bits que salen por la derecha se **pierden**; los huecos de la izquierda se llenan con **0** (para enteros positivos).

Efecto aritmético: `x » k` $= \lfloor x/2^k \rfloor$ (división entera, trunca hacia abajo).

$100/8 = 12.5$ — el desplazamiento trunca y da 12.
En binario: `01100100` → `00001100` = 12. ✓

El desplazamiento a la derecha de **negativos** es *implementation-defined* en C++ hasta C++19. En la práctica, la mayoría de los compiladores extienden con el bit de signo (desplazamiento aritmético), pero no es obligatorio por el estándar. Para evitar sorpresas, trabaja con enteros sin signo (`unsigned int`) cuando desplaces a la derecha.

Operadores lógicos bit a bit

A diferencia de los operadores lógicos `&&` y `||` que operan sobre valores completos, los operadores bit a bit actúan **posición a posición** de forma independiente sobre cada par de bits.

• NOT bit a bit (~)

`~x` invierte cada bit: 0 → 1, 1 → 0.

bit	~bit
0	1
1	0

✔ Recuerda

En complemento a 2, `~x` equivale siempre a $-(x+1)$:

$$\sim 30 = -31, \qquad \sim 0 = -1, \qquad \sim(-1) = 0$$

- ## AND bit a bit (`&`)

  `a & b` produce **1** solo si **ambos** bits correspondientes son 1.

a	b	a & b
1	1	1
1	0	0
0	1	0
0	0	0

Uso típico: *apagar* bits específicos y *consultar* si un bit está encendido.

```
23:        0001 0111
35:        0010 0011
23 & 35:   0000 0011 = 3
```

Solo sobreviven los bits encendidos en **los dos** operandos a la vez (posiciones 0 y 1).

- ## OR bit a bit (`|`)

  `a | b` produce **1** si **al menos uno** de los dos bits es 1.

a	b	a | b
1	1	1
1	0	1
0	1	1
0	0	0

Uso típico: *encender* bits específicos.

```
23:        0001 0111
35:        0010 0011
23 | 35:   0011 0111 = 55
```

El resultado recoge todos los bits encendidos en cualquiera de los dos operandos.

- **XOR bit a bit (^)**

  `a ^ b` produce **1** si los dos bits son **distintos** (eXclusive OR — OR exclusivo).

a	b	a ^ b
1	1	0
1	0	1
0	1	1
0	0	0

Uso típico: *invertir* bits específicos.

```
23:        0001 0111
35:        0010 0011
23 ^ 35:   0011 0100 = 52
```

Los bits donde 23 y 35 **coinciden** se apagan; los que **difieren** se encienden.

Dos identidades del XOR son fundamentales en algoritmos:

- `x ^ x = 0` siempre (cualquier valor XOR consigo mismo es cero).
- `x ^ 0 = x` siempre (XOR con cero deja el valor intacto).

Consecuencia: `(x ^ k) ^ k = x`. Aplicar XOR dos veces con el mismo valor regresa al original. Esta propiedad es la base de varios algoritmos de cifrado simple y de detección de errores.

Combinando operadores

Los operadores de bits se combinan igual que la aritmética normal. El orden de precedencia en C++ (de mayor a menor) es:

Precedencia	Operadores
Mayor	~ (NOT)
	«, » (desplazamientos)
	& (AND)
	^ (XOR)
Menor	| (OR)

Cuando combines operadores, usa paréntesis generosamente. `a & b | c` no siempre hace lo que parece a primera vista.

Paso	Expresión	Resultado
1	`x = 13`	`0000 1101`
2	`x » 2`	`0000 0011` = 3
3	`~(x » 2)`	`...1111 1100` (invierte todo)
4	`y = 39`	`0010 0111`
5	`~(x»2) & y`	`0010 0100` = **36**

El NOT produce un número con casi todos los bits encendidos; el AND final con 39 filtra y deja pasar solo los bits que 39 tiene encendidos.

Ejercicios

• Desplazamientos

1. Sin usar calculadora, calcula el resultado decimal y la representación binaria de 8 bits de cada expresión.

a) `7 « 2`

b) `100 » 2`

c) `1 « 5`

d) `96 » 3`

2. ¿Cuántos desplazamientos a la izquierda necesitas para convertir el valor 3 en 48? Justifica.

3. El valor de `x` es un número entre 1 y 255 (8 bits). Describe qué le ocurre al patrón de bits si aplicas `x » 8`.

• NOT, AND, OR y XOR

4. Calcula los siguientes resultados a mano. Muestra el proceso con la representación de 8 bits de cada operando.

a) `17 & 12`

b) `13 | 12`

c) `17 ^ 12`

d) `~ 12` (expresa el resultado en decimal)

5. El valor `x = 0b10110011`. Evalúa:

a) `x & 0b00001111` (¿qué hace esta operación?)

b) `x | 0b00001111` (¿qué hace esta operación?)

c) `x ^ 0b11111111` (¿qué hace esta operación?)

6. Completa la tabla (todos los valores en 8 bits):

a	b	a & b	a | b
11001100	10101010	?	?
11110000	00001111	?	?
10101010	10101010	?	?

- **Combinando operadores**

7. Evalúa las siguientes expresiones paso a paso, mostrando el resultado binario de 8 bits en cada etapa.

a) `(~ 7) | (1 « 1) | (1 « 3) | (1 « 5)` seguido de `& 29`

b) `(~(13 » 2)) & 39`

8. La expresión `x ^ y ^ y` siempre produce el mismo resultado sin importar el valor de `y`. ¿Cuál es ese resultado y por qué?

9. Dado `n` un entero positivo, ¿qué representa la expresión `n & (n - 1)` en términos de bits? Pruébalo con `n = 12`.

Resumen

✔ Recuerda

- `bitset<N>`: visualiza N bits de un entero. Solo para impresión, no cambia el valor.
- **Literales**: `0b...` (binario), `0x...` (hexadecimal); equivalentes en tiempo de ejecución.
- **Patrones clave**: `0` = todo apagado; `-1` = todo encendido; `1 « k` = solo el bit k encendido ($= 2^k$).
- **Desplazamientos**: `x « k` $= x \times 2^k$; `x » k` $= \lfloor x/2^k \rfloor$.
- **NOT** (`~x`): invierte todos los bits. Equivale a $-(x+1)$.
- **AND** (`&`): 1 si **ambos** bits son 1. Filtra/apaga bits.
- **OR** (`|`): 1 si **al menos uno** es 1. Enciende bits.
- **XOR** (`^`): 1 si los bits son **distintos**. Invierte bits. Propiedades: `x ^ x = 0`; `x ^ 0 = x`.
- **Precedencia** (mayor a menor): `~` $>$ desplazamientos $>$ `&` $>$ `^` $>$ `|`. Usar paréntesis para claridad.